

02 — Compiler Stages: File-by-File, Method-by-Method

Stage 1 — Lexer (`jockey/lexer.py`)

What a lexer does

A lexer (also called a tokeniser or scanner) converts raw source text into a flat list of tokens. A token is the smallest meaningful unit in a language — a keyword, an operator, a literal value, or an identifier.

The lexer does not understand grammar or meaning. It answers one question: "what type of thing is at the current position, and how many characters does it consume?"

The `Lexer` class

Constructor `__init__(source: str):`

- `self.source` — the entire `.jk` file as a single Python string
- `self.pos` — index of the character the lexer is currently looking at (starts at 0)
- `self.line` / `self.column` — track position for error messages (both start at 1)
- `self.tokens` — the list being built, appended to as tokens are found

`_current() → str | None`: Returns `self.source[self.pos]` without moving. Returns `None` if past end of file. Used to check what character we're looking at right now without consuming it.

`_peek(offset=1) → str | None`: Returns the character `offset` positions ahead without advancing. Used to look at the *next* character to distinguish multi-character operators like `:=` from bare `:`. The default offset of 1 means "one ahead of current".

`_advance() → str`: Consumes and returns the current character:

- Increments `self.pos`
- If the consumed character was `\n`, increments `self.line` and resets `self.column` to 1
- Otherwise increments `self.column`

`_emit(token_type, value, line, column)`: Appends a new `Token` to `self.tokens`. `line` and `column` are captured at the start of each token scan, before any advancing, so the token's position points to its first character.

`tokenize()` — the main loop

```
while pos < len(source):
    _scan_one()          ← handles exactly one token
    append EOF token
return self.tokens
```

The loop runs until `pos` reaches the end of the source. Each call to `_scan_one()` is guaranteed to advance `pos` by at least one character (or the program would loop forever — every case either advances or returns).

`_scan_one()` — one token at a time

The method uses a chain of `if` checks. Order matters — checks that would interfere with each other must be in the right sequence.

Step 1 — Skip whitespace: Space, tab, carriage return are not tokens. Advance and return.

Step 2 — Newlines: Newlines also produce no token. Advancing handles the `self.line` increment via `_advance()`. Return.

Step 3 — Comments (##): If the current character is `#` AND the next is also `#`, it's a comment. Advance characters until a newline or EOF. Produce no token.

Step 4 — Multi-character operators (must come before single-char checks):

`:=` (ASSIGN) — if current is `:` and next is `=`, consume both. `<-` (ARROW_L) — if current is `<` and next is `-`, consume both. `->` (ARROW_R) — if current is `-` and next is `>`, consume both.

These *must* be checked before the single-character check. If we checked `:` first, we'd emit a COLON token and then fail on the dangling `=`.

Step 5 — Single-character operators: A lookup dict maps characters to token types. If current character is in the dict, emit it and advance.

Step 6 — String literals (backtick-delimited): If current is ```, consume it (opening delimiter). Then collect characters until another ``` or a newline/EOF:

- A newline inside a string is a lexer error (JOCKY strings cannot span lines)
- EOF before closing backtick is also an error
- On success, emit `STRING` with the collected characters as value (no backticks)

Step 7 — Number literals: If current is a digit, collect digits. During collection, check whether the next character is `.` followed by a digit — if so, it's a float. The `has_dot` flag ensures we only allow one decimal point.

Step 8 — Identifiers and keywords: If current is a letter or `_`, collect alphanumeric characters and underscores. After collecting, check `KEYWORDS.get(word, TokenType.IDENTIFIER)`. If the word is in the keywords dictionary, it gets that token type; otherwise it becomes an IDENTIFIER.

Step 9 — Error: Anything that didn't match any of the above is an unexpected character. Emit an ERROR token (the lexer continues rather than stopping, so all errors are found).

Stage 2 — Parser (`jocky/parser.py`)

What a parser does

The parser takes the flat token list and builds an Abstract Syntax Tree (AST) — a data structure that represents the *nesting and structure* of the program. The parser answers: "given the current token and what can legally come next, what grammatical construct am I looking at?"

JOCKY uses **recursive descent parsing**: each grammar rule has a corresponding Python method. Methods call each other recursively to handle nested constructs.

The **Parser** class

Constructor: Receives the token list, sets `self.pos = 0`.

`_current()` → **Token**: Returns the token at `self.pos` without advancing.

`_peek_type(offset=0)` → **TokenType**: Returns the type of the token `offset` positions ahead. Used to look ahead without consuming.

`_advance()` → **Token**: Returns the current token and advances `self.pos`. Stops at the last token (EOF) to prevent out-of-bounds.

`_expect(token_type)` → **Token**: The critical helper. Checks that the current token matches `token_type`. If yes, advances and returns it. If no, raises **ParseError** with the line number and a description of what was expected vs what was found. This is how "syntax error at line N" messages are produced.

`_match(*types)` → **bool**: Returns True if the current token's type is any of the given types. Does NOT advance. Used for lookahead decisions.

`parse()` — top level

```
while not EOF:
    expect FUNC
    functions.append(_parse_function_def())
return Program(functions)
```

A JOCKY source file is just a sequence of function definitions. The top-level loop only accepts `func` keywords; anything else is a parse error.

`_parse_function_def()`

```
expect FUNC
expect IDENTIFIER → name
expect LPAREN
if not RPAREN:
    parse parameter list
expect RPAREN
expect ARROW_R      (->)
parse type          → return_type
```

```

parse block      → body
return FunctionDef(name, params, return_type, body)

```

The return type comes after `->`. `_parse_type()` checks the current token against the six type keywords and returns the string form (`'num'`, `'dec'`, etc.).

`_parse_statement()` — dispatch

The parser looks at the current token and dispatches:

Current token	Calls
VAR	<code>_parse_var_decl()</code>
CHECK	<code>_parse_if_statement()</code>
LOOP	<code>_parse_loop_statement()</code>
GIVE	<code>_parse_return_statement()</code>
STOP	Returns <code>BreakStatement</code> immediately
SKIP	Returns <code>SkipStatement</code> immediately
IDENTIFIER with next=ARROW_L	<code>_parse_assignment()</code>
Anything else	<code>_parse_expression()</code> wrapped in <code>ExpressionStatement</code>

The assignment case uses lookahead (`_peek_type(1)`) to distinguish:

- `x <- 5` (assignment)
- `x + 5` (expression starting with identifier)

Without the lookahead, both would look like they start with an identifier.

Operator precedence — the call hierarchy

The most elegant part of recursive descent parsing is how operator precedence is handled. Lower-precedence operators are handled by methods higher in the call stack. Each method delegates to the next-higher-precedence method for its operands.

```

_parse_expression()  ← calls _parse_logic()
  _parse_logic()      ← handles 'also', 'or'   (lowest prec.)
    _parse_comparison() ← handles 'is', 'isnt', 'gt', etc.
      _parse_arithmetic() ← handles '+', '-'
        _parse_term()     ← handles '*', '/', 'mod'
          _parse_unary()  ← handles 'flip', unary '-'
            _parse_primary() ← handles atoms      (highest prec.)

```

Why this enforces precedence:

Consider $2 + 3 * 4$. When `_parse_arithmetic` calls `_parse_term()` to get the right operand of `+`, `_parse_term` greedily consumes $3 * 4$ and returns it as a single `BinaryExpression`. So `_parse_arithmetic` sees $2 + (3*4)$, which is correct.

The `_parse_comparison` special case:

Comparisons are non-associative — `a is b is c` is not valid. So `_parse_comparison` uses an `if` check (consuming at most one comparison operator), while `_parse_logic` and `_parse_arithmetic` use `while` loops (allowing chaining like `a + b + c`).

`_parse_primary()` — the atom

This is the base case that ends the recursion. It handles:

- `NUMBER` → `NumberLiteral(int(value))`
- `FLOAT` → `FloatLiteral(float(value))`
- `STRING` → `StringLiteral(value)`
- `YES` → `BoolLiteral(True)`
- `NO` → `BoolLiteral(False)`
- `EMPTY` → `EmptyLiteral()`
- `IDENTIFIER` followed by `(` → `_parse_function_call()`
- `IDENTIFIER` alone → `Identifier(name)`
- `(` → consume, parse sub-expression, expect `)`, return sub-expression

Anything else → `ParseError`.

Stage 3 — Semantic Analyser (`jocky/semantic.py`)

What semantic analysis does

Parsing only checks structure. Semantic analysis checks *meaning*:

- Does every variable you use exist?
- Does every function you call exist, with the right number of arguments?
- Are types compatible?
- Does the return type match the declared function signature?

The two-pass structure

Pass 1 — Register all function signatures:

```
for func in program.functions:
    symtab.declare_function(func.name, param_types, return_type)
```

This runs before any body is analysed. Without it, a function defined later in the file could not be called by one defined earlier (forward reference). By registering all signatures first, JOCKY allows any function to call any other function regardless of order.

Pass 2 — Analyse function bodies:

```
for func in program.functions:
    _visit_function(func)
```

`_seed_stdlib()`

Called in `__init__`. Inserts all 14 stdlib function signatures into the symbol table before any user code is analysed. This is why `report()`, `procs_list()`, etc. are available without declaration.

```
STDLIB_SIGNATURES = {
    'report':      ([ 'text' ],      'nothing'),
    'procs_list': ([ ],             'raw'),
    'proc_count': ([ 'raw' ],        'num'),
    ...
}
```

`_visit_expr()` — type inference

Every expression node goes through this method, which returns a type string and annotates the node's `inferred_type` field.

Literals: Trivial — `NumberLiteral` always returns `'num'`, `StringLiteral` always `'text'`, etc.

Identifier lookup: Calls `symtab.lookup_var(name)`. Returns the symbol's declared type.

Binary expressions:

- Arithmetic (`+`, `-`, `*`, `/`, `mod`): both operands must be numeric. If either is `dec`, result is `dec`. Otherwise `num`.
- Comparison (`is`, `isnt`): types must match (or both be numeric). Result is always `flag`.
- Ordering (`gt`, `lt`, `gte`, `lte`): both must be numeric. Result is `flag`.
- Logic (`also`, `or`): both must be `flag`. Result is `flag`.

Function calls:

- Looks up the function in the symbol table
- Checks argument count matches
- Each argument is itself visited (recursive — `_visit_expr` for each arg)
- Returns the function's declared return type

Error collection

Instead of raising an exception on the first error, the analyser collects errors into `self.errors: list[str]`. At the end of `analyze()`:

```
if self.errors:
    raise SemanticError("Semantic analysis failed:\n" + "\n".join(self.errors))
```

This means you see ALL your type errors in one run, not just the first one.

`_check_assignable(target, source, context, line)`

Called when assigning a value to a variable or returning from a function. Rules:

- `target == source` → always OK
 - Both in `{'num', 'dec'}` → OK (numeric coercion, handled later in codegen)
 - `target == 'raw'` and `source == 'nothing'` → OK (null assignment to pointer)
 - Anything else → error
-

Stage 4 — Code Generator (`jocky/codegen.py`)

What code generation does

Takes the semantically-annotated AST and emits LLVM IR instructions using the `llvmlite.ir` API. Every AST node becomes one or more IR instructions.

The `CodeGenerator` class state

```
self.module      # ir.Module - the container for everything
self.builder     # ir.IRBuilder - emits instructions; changes per function
self.cur_func    # ir.Function - the function being generated
self.value_table # dict[str, alloca] - maps var name to its stack slot
self.func_table  # dict[str, ir.Function] - all declared functions
self.break_stack # stack of BasicBlocks for 'stop' targets
self.continue_stack # stack of BasicBlocks for 'skip' targets
self._str_idx    # counter for unique global string names
```

The `value_table` is reset for each function (local variables don't persist across functions). The `func_table` persists for the entire module (functions are global).

`_declare_stdlib()`

Called in `__init__`. For each stdlib function, creates an `ir.Function` with `External` linkage — a declaration, not a definition. This tells LLVM "this function exists somewhere; the linker (or JIT resolver) will find it."

```
fn_type = ir.FunctionType(void_type, [i8_pointer_type])
fn = ir.Function(self.module, fn_type, name='report')
```

Also declares `strcmp` — needed for text comparison.

Two-pass generation

Pass 1 — `_declare_user_func()`: For each user-defined function, creates an `ir.Function` shell (no body yet) and stores it in `func_table`. This allows forward calls (function A can call function B even if B is declared later in the file).

Pass 2 — `_gen_function()`: Generates the body of each function.

`_gen_function()` — how a function becomes IR

```
fn = func_table[node.name]
Create 'entry' basic block
Create IRBuilder positioned at entry block

For each parameter:
    alloca = builder.alloca(type, name)    ← allocate stack space
    builder.store(fn.args[i], alloca)      ← store arg value there
    value_table[param.name] = alloca      ← remember the slot

Generate the body block

If block doesn't end with a terminator (ret/br):
    Add implicit ret_void() or ret(0)
```

Every parameter gets its own `alloca` (stack slot). The parameter value is stored into the slot immediately. When the parameter is referenced later, it is loaded from the slot. This pattern works for variables too — every `var` declaration creates an `alloca`.

Why `alloca` / `load` / `store` instead of direct SSA?

LLVM IR is SSA (Static Single Assignment) — every value is assigned exactly once and never changes. This seems incompatible with mutable variables (`x <- x + 1`). The solution: store variables in *memory* (on the stack), not as IR values.

```
%x = alloca i64          ; allocate 8 bytes on the stack
store i64 5, i64* %x     ; write 5 into those bytes
%x.1 = load i64, i64* %x ; read current value out
%x.2 = add i64 %x.1, 1   ; compute x + 1
store i64 %x.2, i64* %x  ; write new value back
```

The `alloca` pointer `%x` is used multiple times (reads and writes), but it's always the same pointer — SSA is satisfied because the *pointer* itself never changes, only the *memory it points to*.

LLVM's `mem2reg` optimisation pass promotes these allocas to actual SSA registers when it can, so the overhead disappears in optimised builds.

Control flow — if statements

```
check (cond) { ... } otherwise { ... }
```

Generates four basic blocks:

```
current_block:
    cond_val = gen_expr(condition)
    cbranch cond_val → then_bb, else_bb

then_bb:
    gen_block(then_body)
    branch → merge_bb

else_bb:
    gen_block(else_body)    ← if no 'otherwise', just branch merge_bb
    branch → merge_bb

merge_bb:
    (execution continues here)
```

`cbranch` is a conditional branch: it takes a condition (i1) and two target blocks. `branch` is unconditional. Every basic block must end with a terminator — either a branch or a return.

Control flow — loops

```
loop (cond) { ... }
```

Generates three basic blocks:

```
current_block:
    branch → loop_cond

loop_cond:
    cond_val = gen_expr(condition)
    cbranch cond_val → loop_body, loop_exit

loop_body:
    gen_block(body)
    branch → loop_cond    ← back-edge (what makes it a loop)
```

```
loop_exit:
    (execution continues here)
```

The `break_stack` holds `loop_exit` blocks. When `stop` is encountered:

```
self.builder.branch(self.break_stack[-1])
```

The `continue_stack` holds `loop_cond` blocks. When `skip` is encountered:

```
self.builder.branch(self.continue_stack[-1])
```

String generation — `_gen_string()`

Every string literal becomes a global constant in the IR:

```
encoded = (text + '\x00').encode('utf-8') # null-terminate
arr_type = ir.ArrayType(self.i8, len(encoded))
gv       = ir.GlobalVariable(self.module, arr_type, name=f'.jk_str.{idx}')
gv.global_constant = True
gv.linkage         = 'internal'
gv.initializer     = ir.Constant(arr_type, bytearray(encoded))
```

Then a GEP (Get Element Pointer) instruction extracts an `i8*` pointer to element [0]:

```
zero = ir.Constant(self.i32, 0)
ptr  = builder.gep(gv, [zero, zero], inbounds=True, name='strptr')
```

The GEP `[0, 0]` means: "from the global (which is a pointer to the array), first dereference (index 0 into the globals list), then index to element 0 of the array." This gives an `i8*` pointing to the first byte of the string — the C `char*` that `report()` and `strcmp()` expect.

String comparison

When the `is` or `isnt` operator is applied to `text` type values, the operands are `i8*` pointers. Comparing two pointers with `icmp eq` compares their *addresses*, not their contents. Two different string literals with the same content would have different addresses and compare as not equal.

Instead, the codegen detects pointer-type operands:

```

if op in ('is', 'isnt') and isinstance(left.type, ir.PointerType):
    cmp = builder.call(func_table['strcmp'], [left, right], 'strcmpres')
    zero = ir.Constant(i32, 0)
    eq = builder.icmp_signed('==', cmp, zero, 'streq')
    return eq if op == 'is' else builder.not_(eq, 'strne')

```

`strcmp` returns 0 when strings are equal, non-zero otherwise. Comparing the result to 0 produces the `i1` boolean.

Type coercion — `_coerce()`

Called before storing a value into a typed alloca or passing it as a typed argument. Handles:

Situation	IR instruction
<code>num</code> → <code>dec</code>	<code>sitofp</code> (signed int to float)
<code>dec</code> → <code>num</code>	<code>fptosi</code> (float to signed int)
<code>i1</code> → <code>i64</code>	<code>zext</code> (zero-extend)
<code>i64</code> → <code>i1</code>	<code>trunc</code> (truncate)
Same type	no-op

Stage 5 — Obfuscation Passes (`jocky/passes.py`)

Architecture

`ObfuscationPasses` wraps the `ir.Module` and applies three mutations in sequence. Each mutation changes the IR in-place; `run_all()` returns the modified module.

Obfuscation only runs in binary output mode (when generating `.o` files). It is explicitly skipped during JIT execution because:

- The JIT mode's Python callbacks receive strings at the point the generated code calls them
- If the strings are encrypted in the IR, the bytes passed to `report()` are encrypted bytes, not the original text
- Without a runtime decryptor, output would be garbled

Pass 1 — Polymorphic Build-ID

```

i8      = ir.IntType(8)
arr16   = ir.ArrayType(i8, 16)
marker  = ir.GlobalVariable(module, arr16, name='_jocky_build_id')
marker.global_constant = True
marker.linkage          = 'internal'

```

```
build_id = bytearray(random.randint(0, 255) for _ in range(16))
marker.initializer = ir.Constant(arr16, build_id)
```

Adds a 16-byte global constant filled with cryptographically random bytes. Because `random.randint(0, 255)` produces different values every run, the final `.o` and `.exe` contain different bytes → different SHA-256 hash.

The global is `internal` (not exported) so it doesn't conflict with anything. It is `constant` so it lands in the read-only data section of the binary (`.rdata` on Windows). It is never referenced by code — it is pure data injected to make the binary unique.

Pass 2 — String XOR Encryption

Finds every global whose name starts with `.jk_str`:

```
string_globals = [
    gv for gv in module.global_values
    if (isinstance(gv, ir.GlobalVariable)
        and gv.name.startswith('.jk_str')
        and isinstance(gv.type.pointee, ir.ArrayType)
        and isinstance(gv.type.pointee.element, ir.IntType)
        and gv.type.pointee.element.width == 8)
]
```

For each string:

1. Extract the bytes from `gv.initializer.constant`
2. Generate a random key byte (1–255, never 0 to ensure the null terminator is also encrypted)
3. XOR every byte with the key: `encrypted = bytearray(b ^ key for b in raw)`
4. Replace the initializer: `gv.initializer = ir.Constant(arr_type, encrypted)`
5. Create a companion `.jk_str.N.key` global holding the key byte

```
Before: @".jk_str.0" = internal constant [19 x i8] c"Hello from JOCKY!\00"
After:  @".jk_str.0" = internal constant [19 x i8] c"\3f\1a\7b..."
        @".jk_str.0.key" = internal constant [1 x i8] c"\4a"
```

An AV string scanner finds no readable text. The binary appears to contain only random noise.

Pass 3 — Entropy Noise

```
noise = ir.GlobalVariable(module, ir.IntType(64), name='_jocky_entropy')
noise.initializer = ir.Constant(ir.IntType(64), random.randint(0, 2**63 - 1))
```

A random 64-bit integer. Adds more variation to the binary — different bytes at a different offset than the build-ID. Further distinguishes binaries structurally beyond just the random header.

`compute_hash(file_path) → str`

A utility function that reads any file in 64 KB chunks and computes its SHA-256:

```
sha256 = hashlib.sha256()
with open(file_path, 'rb') as f:
    for chunk in iter(lambda: f.read(65536), b''):
        sha256.update(chunk)
return sha256.hexdigest()
```

Used in `compiler.py` to print the hash of the compiled `.o` and `.exe` files for the polymorphism demonstration.